

INSTITUTO POLITÉCNICO NACIONAL

Escuela Superior de Cómputo (ESCOM / UPIIZ)

Ingeniería en Sistemas Computacionales

TEMA

Constante de Kaprekar

Asignatura: Desarrollo de Aplicaciones Móviles Nativas

Tarea: Tarea 6: Constante de Kaprekar

Alumno: Angel Frausto Robles

Grupo: 7CM4

Fecha: 12 de septiembre de 2026

INTRODUCCIÓN

Esta tarea pide construir una aplicación Android que reciba un número de 3 o 4 dígitos y verifique si, al aplicarle repetidamente la rutina de Kaprekar, llega a la constante correspondiente: 495 para números de 3 dígitos o 6174 para números de 4 dígitos.

Objetivos:

- Implementar la rutina de Kaprekar (ordenar dígitos, restar el menor del mayor) en Java.
- Detectar el caso especial de los números con todos sus dígitos iguales, donde la rutina llega a 0 en lugar de a la constante.
- Mostrar los pasos intermedios, no solo si el número llega o no a la constante.

DESARROLLO

1. Entorno de trabajo

Igual que en la Tarea 5, el proyecto se hizo en Java con el SDK clásico de Android: `MainActivity` extiende `android.app.Activity` y la interfaz usa vistas nativas (`EditText`, `Button`, `TextView`), sin `AndroidX` ni `ConstraintLayout`.

2. Interfaz (activity_main.xml)

El diseño es el mismo patrón que la Tarea 5: título, campo de texto, botón "Verificar" y un `TextView` con los pasos de la rutina, todo dentro de un `ScrollView`.

```
<ScrollView ...>
  <LinearLayout android:orientation="vertical" ...>
    <TextView android:text="@string/xtitulo" />
    <EditText android:id="@+id/xnumero" android:inputType="number"
      android:hint="@string/xdato" />
    <Button android:id="@+id/xboton" android:text="@string/xaceptar" />
    <TextView android:id="@+id/xresultado" android:fontFamily="monospace" />
  </LinearLayout>
</ScrollView>
```

3. Lógica (MainActivity.java)

Primero se valida que el número tenga exactamente 3 o 4 dígitos y se fija el objetivo correspondiente (495 o 6174). Cada paso de la rutina ordena los dígitos de forma ascendente y descendente con `Arrays.sort` y resta el menor del mayor:

```
private int pasoKaprekar(int numero, int digitos) {
    char[] cifras = String.format("%0" + digitos + "d", numero).toCharArray();

    char[] ascendente = cifras.clone();
    Arrays.sort(ascendente);

    char[] descendente = cifras.clone();
    Arrays.sort(descendente);
    invertir(descendente);

    int menor = Integer.parseInt(new String(ascendente));
    int mayor = Integer.parseInt(new String(descendente));
    return mayor - menor;
}
```

El ciclo principal repite este paso hasta que el número llega al objetivo o hasta que llega a 0, que es lo que pasa cuando los cuatro (o tres) dígitos son iguales: al restar el mismo número ordenado de las dos formas, el resultado siempre es 0, y ahí se queda para siempre porque 0 no es ni 495 ni 6174.

4. Pruebas en el emulador Android

Se probaron tres casos: uno de 4 dígitos, uno de 3 dígitos y el caso especial de dígitos repetidos:

Número	Dígitos	Resultado	Pasos
3529	4	Llega a 6174	5
753	3	Llega a 495	3

111	3	No llega (dígitos repetidos)	Llega a 0 en 1 paso
-----	---	------------------------------	---------------------

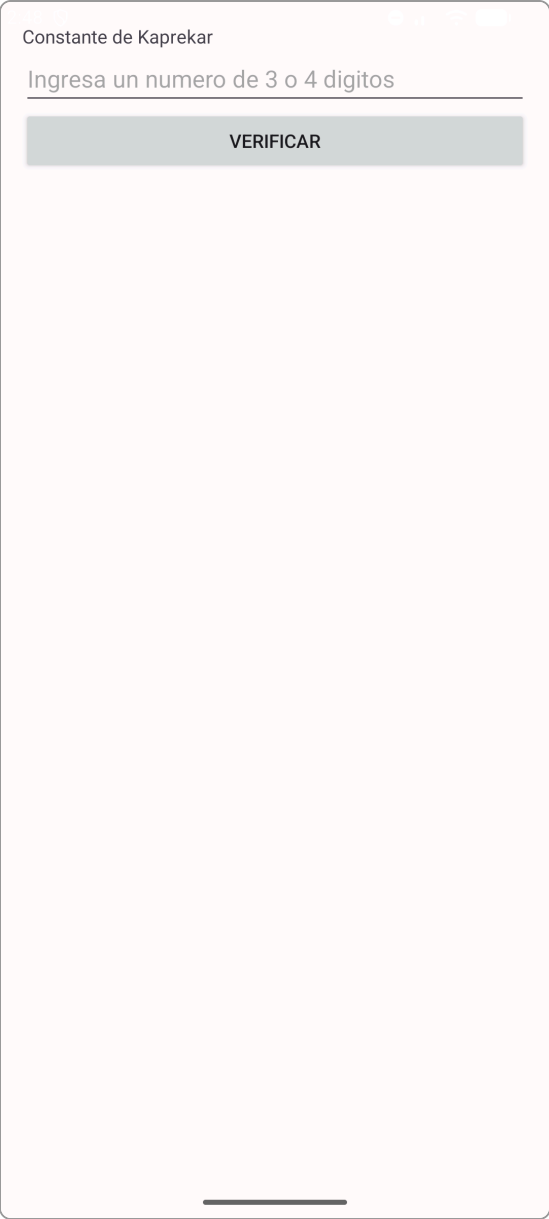


Figura 1. Pantalla inicial de la aplicación.

Constante de Kaprekar

3529

VERIFICAR

Numero analizado: 3529

SI llega a la constante de Kaprekar (6174).

Pasos:
3529 -> 7173 -> 6354 -> 3087 -> 8352 -> 6174

Cantidad de pasos: 5

Figura 2. Número 3529: llega a 6174 en 5 pasos.

Constante de Kaprekar

753

VERIFICAR

Numero analizado: 753

SI llega a la constante de Kaprekar (495).

Pasos:
753 -> 396 -> 594 -> 495

Cantidad de pasos: 3

Figura 3. Número 753: llega a 495 en 3 pasos.

Constante de Kaprekar

111

VERIFICAR

Numero analizado: 111

NO llega a la constante (digitos repetidos, la rutina llega a 0).

Pasos:
111 -> 0

Figura 4. Número 111: dígitos repetidos, la rutina llega a 0.

CONCLUSIONES

Lo que más se me quedó de esta tarea es que la constante de Kaprekar no depende del número que uno elija: cualquier número de 4 dígitos que no tenga sus cuatro dígitos iguales termina en 6174, y con números de 3 dígitos pasa lo mismo con 495. Probarlo con 3529 y 753 y ver que ambos convergen en pocos pasos ayuda a confiar en la rutina, pero el caso de 111 fue el que más me sirvió para entender por qué hace falta ese caso especial en el código: sin él, el ciclo se quedaría comparando 0 contra el objetivo para siempre.

BIBLIOGRAFÍA

- Cifuentes A., Alejandro. *DAMN Kotlin Practica4* (documento de la materia donde se planteó por primera vez la constante de Kaprekar).
- Android Developers. *Activity, Views and Layouts*. <https://developer.android.com/guide>
- Wikipedia. *Kaprekar's constant*. [https://en.wikipedia.org/wiki/6174_\(number\)](https://en.wikipedia.org/wiki/6174_(number))